

R AND PYTHON CODE 15-7-2026

```
1) import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# -----
# Generate Sample Dataset
# -----
np.random.seed(10)

n = 150

data = pd.DataFrame({
    "Sensor": range(1, n+1),
    "PM2.5": np.random.randint(10, 250, n),
    "NO2": np.random.randint(5, 180, n),
    "CO": np.random.uniform(0.1, 5.0, n),
    "Temperature": np.random.randint(20, 45, n),
    "Humidity": np.random.randint(30, 95, n),
    "X": np.random.randint(1, 21, n),
    "Y": np.random.randint(1, 21, n)
})

print(data.head())

# -----
# Sequential Color Palette
# -----
plt.figure(figsize=(8,6))
plt.scatter(data["X"], data["Y"],
            c=data["PM2.5"],
            cmap="Greens",
            s=120)

plt.colorbar(label="PM2.5")
plt.title("Sequential Color Palette (Greens)")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.grid(True)
plt.show()

# -----
# Diverging Color Palette
# -----
plt.figure(figsize=(8,6))
plt.scatter(data["X"], data["Y"],
            c=data["PM2.5"],
            cmap="coolwarm",
            s=120)

plt.colorbar(label="PM2.5")
plt.title("Diverging Color Palette (Coolwarm)")
```

R AND PYTHON CODE 15-7-2026

```
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.grid(True)
plt.show()

# -----
# Qualitative Color Palette
# -----

colors = sns.color_palette("Set2", 5)

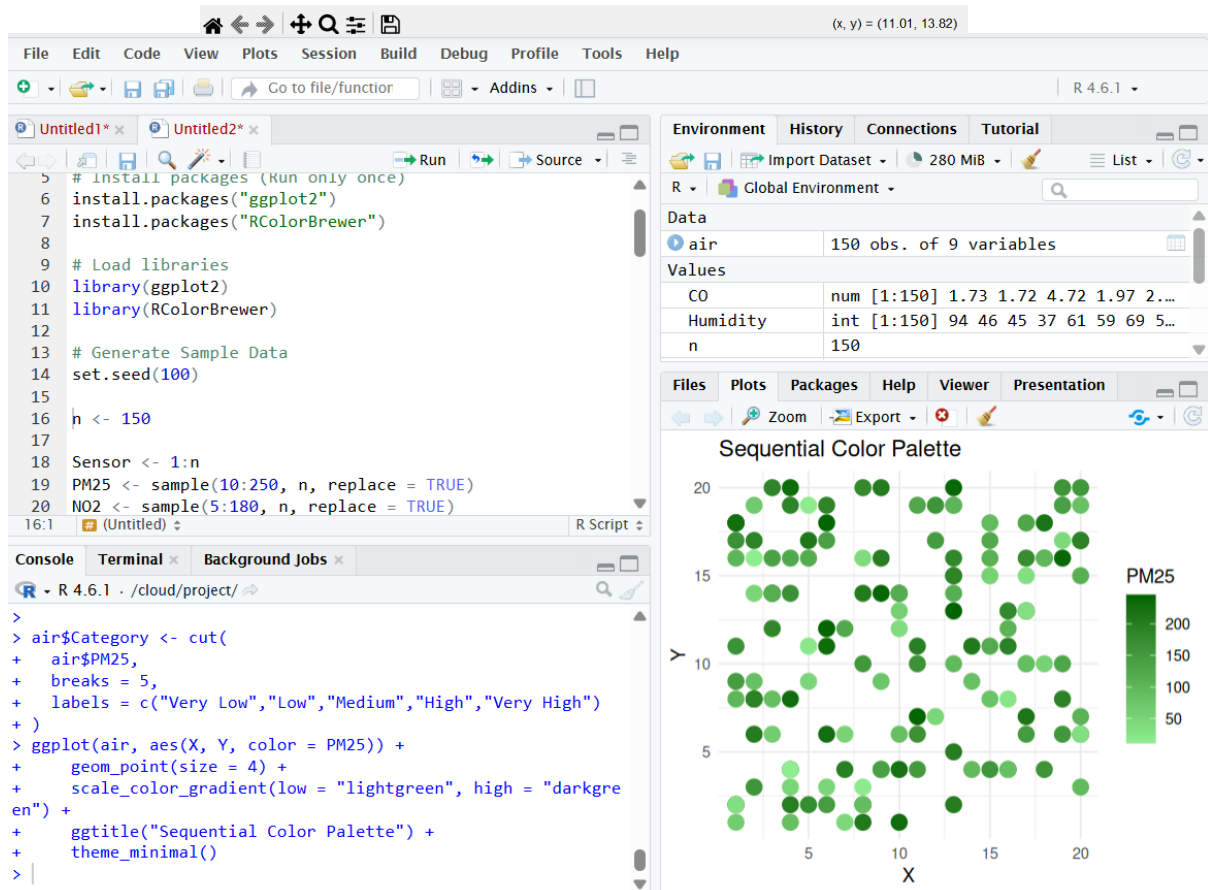
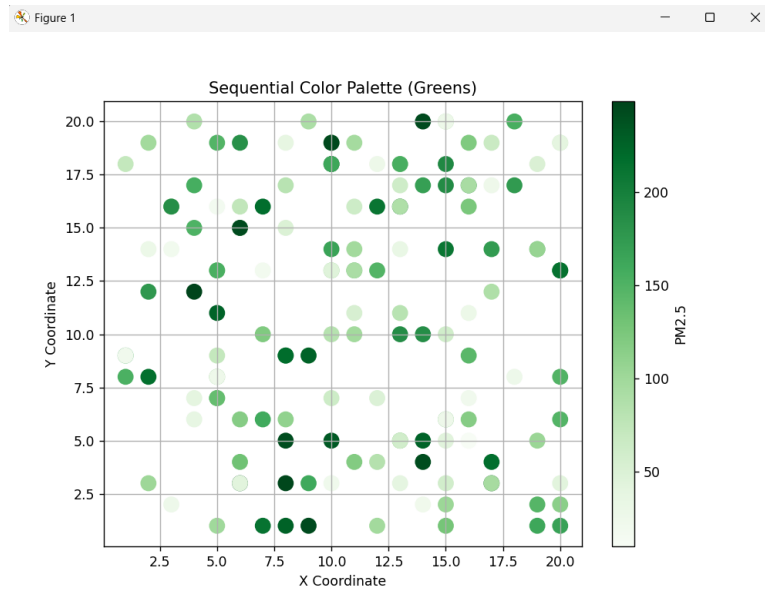
category = pd.cut(data["PM2.5"],
                  bins=5,
                  labels=False)

plt.figure(figsize=(8,6))

for i in range(5):
    subset = data[category == i]
    plt.scatter(subset["X"],
                subset["Y"],
                color=colors[i],
                s=120,
                label=f"Group {i+1}")

plt.title("Qualitative Palette (Set2)")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.legend()
plt.grid(True)
plt.show()
```

R AND PYTHON CODE 15-7-2026



2)

#=====

EXPERIMENT 2

Earthquake Risk Analysis

#=====

Install package (Run only once)

R AND PYTHON CODE 15-7-2026

```
if (!require(ggplot2)) {  
  install.packages("ggplot2")  
  library(ggplot2)  
} else {  
  library(ggplot2)  
}  
  
#-----  
# Generate Sample Earthquake Data  
#-----  
set.seed(123)  
  
n <- 1000  
  
Magnitude <- pmin(rexp(n, rate = 1/1.5) + 0.5, 9.5)  
Depth <- sample(1:700, n, replace = TRUE)  
Latitude <- runif(n, -90, 90)  
Longitude <- runif(n, -180, 180)  
Aftershock <- sample(0:60, n, replace = TRUE)  
  
earthquake <- data.frame(  
  Magnitude,  
  Depth,  
  Latitude,  
  Longitude,  
  Aftershock  
)  
  
print(head(earthquake))  
  
#=====
```

1. Scatter Plot - Linear Scale

```
#=====
```

R AND PYTHON CODE 15-7-2026

```
p1 <- ggplot(earthquake,
             aes(x = Magnitude,
                 y = Depth,
                 color = Aftershock)) +
  geom_point(size = 2) +
  labs(
    title = "Earthquake Distribution - Linear Scale",
    x = "Magnitude",
    y = "Depth (km)"
  ) +
  theme_minimal()

print(p1)

#=====

# 2. Scatter Plot - Log Scale

#=====

p2 <- ggplot(earthquake,
             aes(x = Magnitude,
                 y = Depth,
                 color = Aftershock)) +
  geom_point(size = 2) +
  scale_y_log10() +
  labs(
    title = "Earthquake Distribution - Log Scale",
    x = "Magnitude",
    y = "Depth (Log Scale)"
  ) +
  theme_minimal()

print(p2)

#=====
```

R AND PYTHON CODE 15-7-2026

3. Histogram - Linear Scale

```
#=====
```

```
p3 <- ggplot(earthquake,
             aes(x = Magnitude)) +
  geom_histogram(
    bins = 30,
    fill = "skyblue",
    color = "black"
  ) +
  labs(
    title = "Magnitude Distribution - Linear",
    x = "Magnitude",
    y = "Frequency"
  ) +
  theme_minimal()

print(p3)
```

```
#=====
```

4. Histogram - Log Scale

```
#=====
```

```
p4 <- ggplot(earthquake,
             aes(x = Magnitude)) +
  geom_histogram(
    bins = 30,
    fill = "orange",
    color = "black"
  ) +
  scale_y_log10() +
  labs(
    title = "Magnitude Distribution - Log Scale",
    x = "Magnitude",
```

R AND PYTHON CODE 15-7-2026

```
y = "Frequency (Log)"

)+

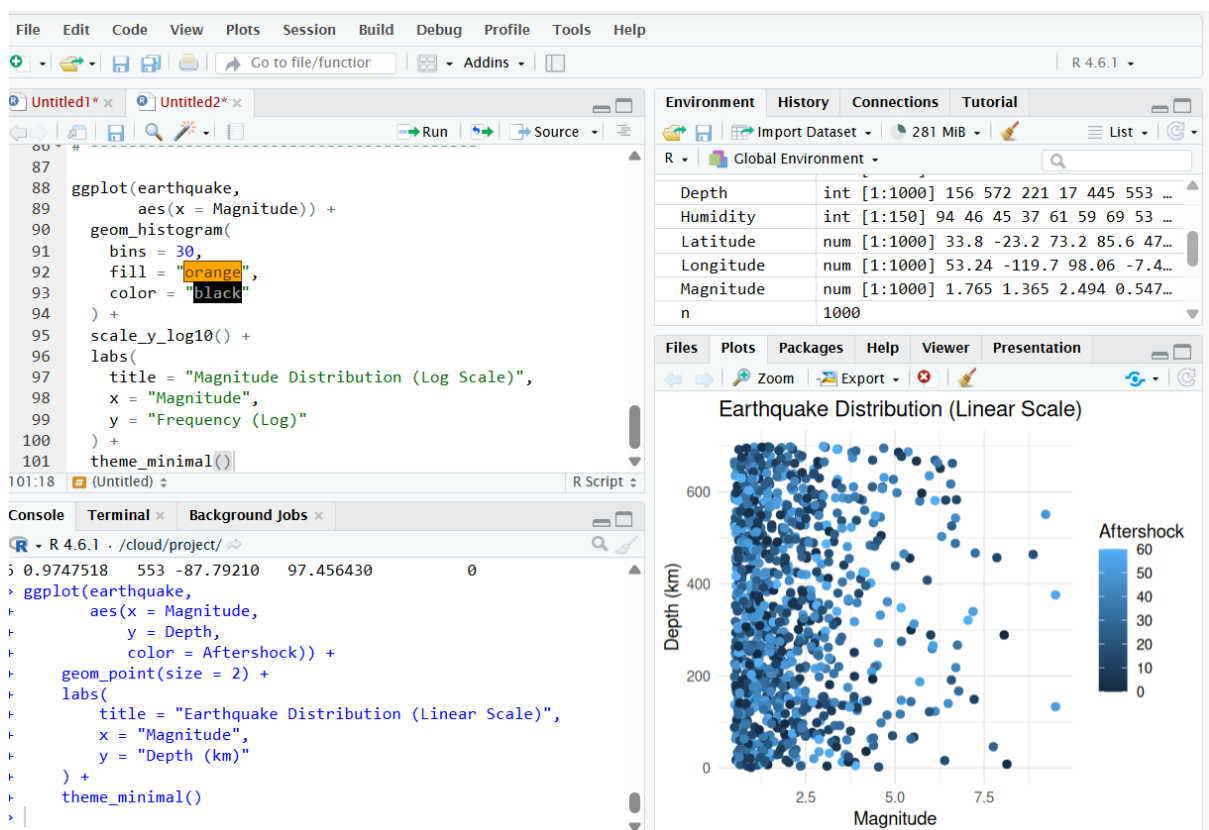
theme_minimal()

print(p4)

#=====

# End of Program

#=====
```



```
3)

#=====

# EXPERIMENT 3

# Wind Direction Analysis for Airport Operations

#=====

# Install packages (Run only once)

if (!require(ggplot2)) install.packages("ggplot2")

library(ggplot2)
```

R AND PYTHON CODE 15-7-2026

```
#-----  
# Generate Sample Data  
#-----  
  
set.seed(123)  
  
n <- 1000  
  
WindSpeed <- runif(n, 0, 25)  
WindDirection <- runif(n, 0, 360)  
  
wind <- data.frame(  
  WindSpeed,  
  WindDirection  
)  
  
print(head(wind))  
  
#=====
```

1. Cartesian Scatter Plot

```
#=====
```

p1 <- ggplot(wind,
 aes(x = 1:n,
 y = WindDirection,
 color = WindSpeed)) +
geom_point(size = 2) +
labs(
 title = "Wind Direction - Cartesian Plot",
 x = "Observation",
 y = "Wind Direction (Degrees)"
) +
theme_minimal()

R AND PYTHON CODE 15-7-2026

```
print(p1)
```

```
#=====
```

```
# 2. Polar Coordinate Plot
```

```
#=====
```

```
p2 <- ggplot(wind,
             aes(x = factor(round(WindDirection/30)*30),
                 fill = WindSpeed)) +
  geom_bar() +
  coord_polar() +
  labs(
    title = "Wind Rose (Polar Coordinate Plot)",
    x = "Direction",
    y = "Frequency"
  ) +
  theme_minimal()
```

```
print(p2)
```

```
#=====
```

```
# 3. Wind Speed Histogram
```

```
#=====
```

```
p3 <- ggplot(wind,
             aes(x = WindSpeed)) +
  geom_histogram(
    bins = 20,
    fill = "skyblue",
    color = "black"
  ) +
  labs(
    title = "Wind Speed Distribution",
```

R AND PYTHON CODE 15-7-2026

```
x = "Wind Speed (m/s)",
y = "Frequency"

) +
theme_minimal()

print(p3)

#=====

# 4. Wind Direction Histogram

#=====

p4 <- ggplot(wind,
             aes(x = WindDirection)) +
geom_histogram(
  bins = 12,
  fill = "orange",
  color = "black"
) +
labs(
  title = "Wind Direction Distribution",
  x = "Direction (Degrees)",
  y = "Frequency"
) +
theme_minimal()

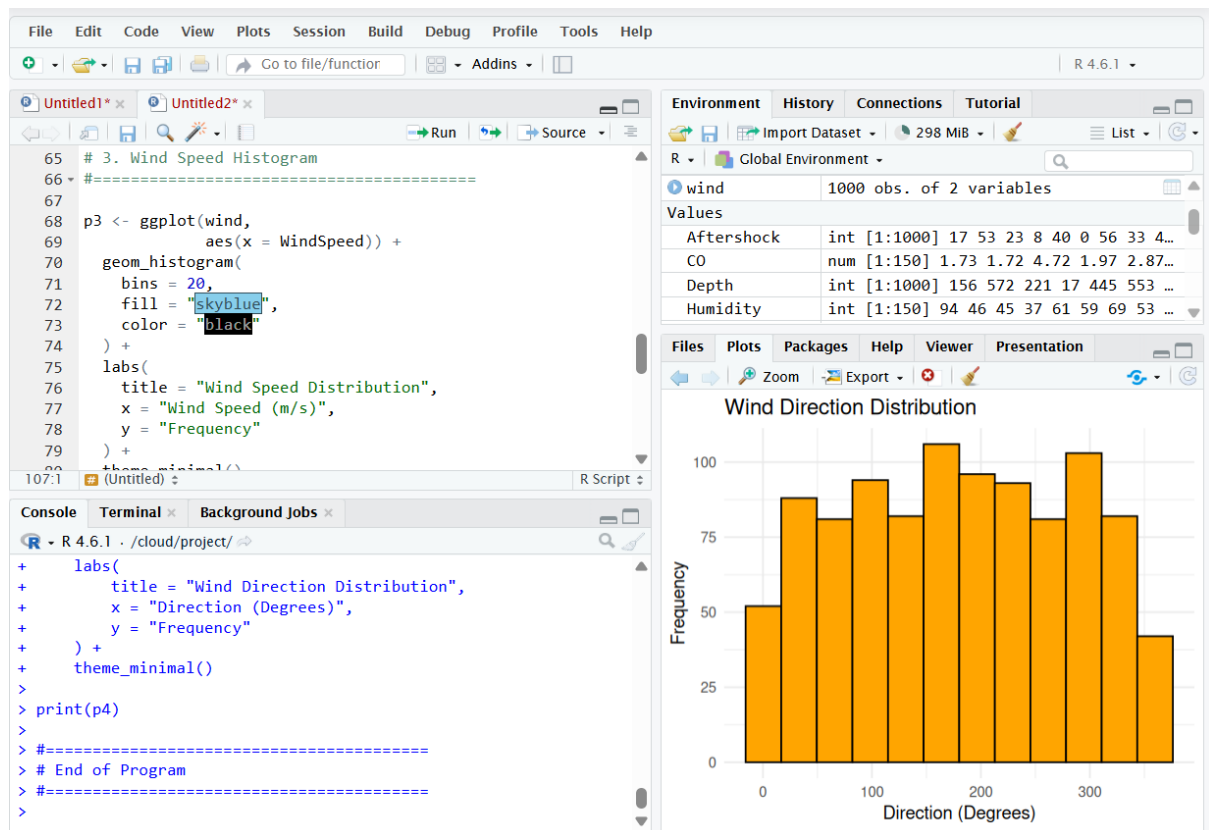
print(p4)

#=====

# End of Program

#=====
```

R AND PYTHON CODE 15-7-2026



4)

#=====

EXPERIMENT 4

Hospital Patient Monitoring Dashboard

#=====

Install packages (Run only once)

```
if (!require(ggplot2)) install.packages("ggplot2")
```

```
library(ggplot2)
```

#-----

Generate Sample Patient Data

#-----

```
set.seed(100)
```

```
n <- 200
```

R AND PYTHON CODE 15-7-2026

```
Time <- 1:n

HeartRate <- sample(50:140, n, replace = TRUE)

Oxygen <- sample(85:100, n, replace = TRUE)

SystolicBP <- sample(90:180, n, replace = TRUE)

Respiration <- sample(10:35, n, replace = TRUE)

Temperature <- round(runif(n, 35, 40),1)


patient <- data.frame(

  Time,

  HeartRate,

  Oxygen,

  SystolicBP,

  Respiration,

  Temperature

)


print(head(patient))


#=====

# Heart Rate Dashboard

#=====


patient$HR_Status <- ifelse(patient$HeartRate < 60 | patient$HeartRate > 100,

  "Abnormal","Normal")


p1 <- ggplot(patient,

  aes(Time, HeartRate, color = HR_Status)) +

  geom_point(size = 2) +

  scale_color_manual(values = c("Normal"="green",

    "Abnormal"="red")) +

  ggtitle("Heart Rate Monitoring") +

  theme_minimal()


print(p1)
```

R AND PYTHON CODE 15-7-2026

```
#=====
```

```
# Oxygen Saturation Dashboard
```

```
#=====
```

```
patient$O2_Status <- ifelse(patient$Oxygen < 95,  
                             "Low", "Normal")
```

```
p2 <- ggplot(patient,  
              aes(Time, Oxygen, color = O2_Status)) +  
  geom_point(size = 2) +  
  scale_color_manual(values = c("Normal"="blue",  
                                "Low"="red")) +  
  ggtitle("Oxygen Saturation") +  
  theme_minimal()
```

```
print(p2)
```

```
#=====
```

```
# Blood Pressure Dashboard
```

```
#=====
```

```
patient$BP_Status <- ifelse(patient$SystolicBP > 140,  
                             "High", "Normal")
```

```
p3 <- ggplot(patient,  
              aes(Time, SystolicBP, color = BP_Status)) +  
  geom_line() +  
  scale_color_manual(values = c("Normal"="green",  
                                "High"="red")) +  
  ggtitle("Blood Pressure") +  
  theme_minimal()
```

```
print(p3)
```

R AND PYTHON CODE 15-7-2026

```
#=====

# Temperature Dashboard

#=====

patient$Temp_Status <- ifelse(patient$Temperature > 37.5,
                              "High", "Normal")

p4 <- ggplot(patient,
             aes(Time, Temperature, color = Temp_Status)) +
  geom_line() +
  scale_color_manual(values = c("Normal"="blue",
                              "High"="red")) +
  ggtitle("Body Temperature") +
  theme_minimal()

print(p4)

#=====

# Respiration Dashboard

#=====

patient$Resp_Status <- ifelse(patient$Respiration > 20,
                              "High", "Normal")

p5 <- ggplot(patient,
             aes(Time, Respiration, color = Resp_Status)) +
  geom_line() +
  scale_color_manual(values = c("Normal"="green",
                              "High"="red")) +
  ggtitle("Respiration Rate") +
  theme_minimal()

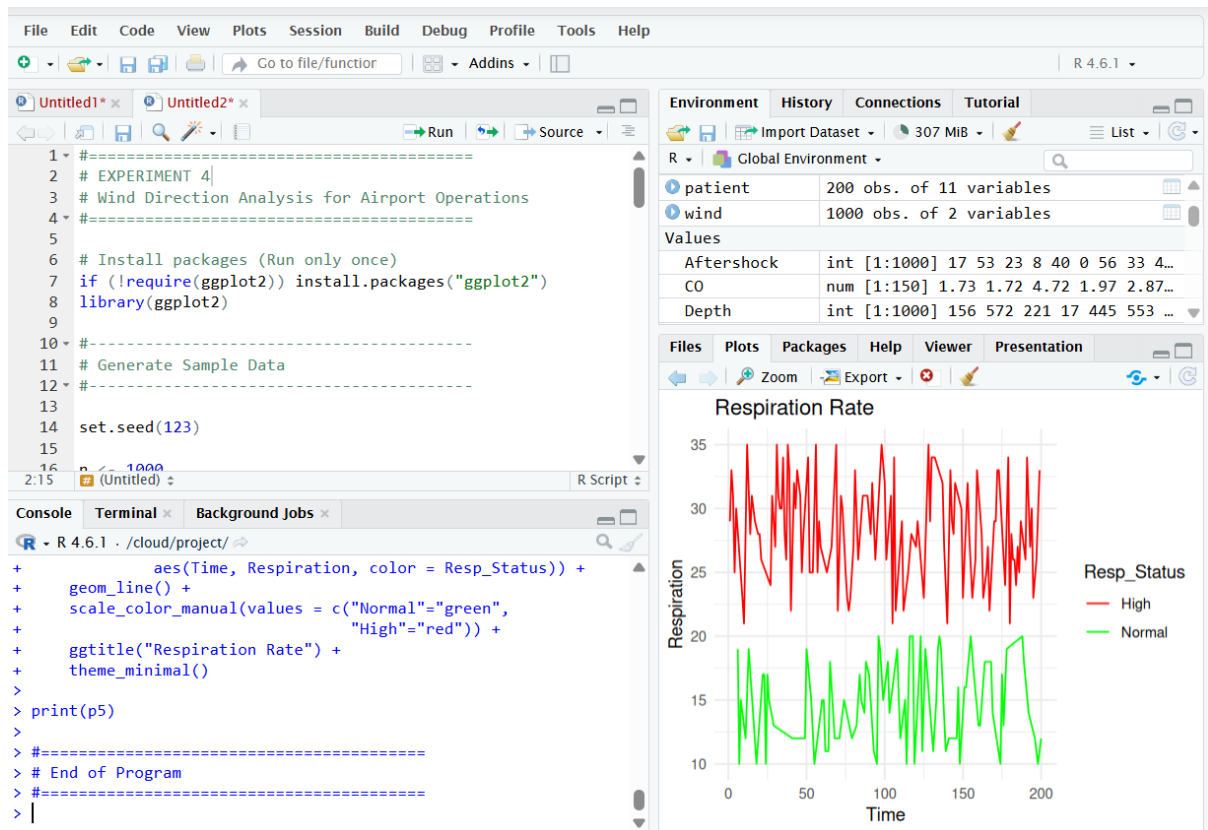
print(p5)
```

R AND PYTHON CODE 15-7-2026

```
#=====
```

```
# End of Program
```

```
#=====
```



5)

```
#=====
```

```
# EXPERIMENT 5
```

```
# Global Climate Change Study
```

```
#=====
```

```
# Install packages (Run only once)
```

```
if (!require(ggplot2)) install.packages("ggplot2")
```

```
if (!require(RColorBrewer)) install.packages("RColorBrewer")
```

```
library(ggplot2)
```

```
library(RColorBrewer)
```

```
#-----
```

```
# Generate Sample Data
```

R AND PYTHON CODE 15-7-2026

```
#-----
```

```
set.seed(123)
```

```
countries <- paste("Country",1:180)
```

```
years <- 1980:2025
```

```
climate <- expand.grid(
```

```
  Country = countries,
```

```
  Year = years
```

```
)
```

```
climate$Temperature_Anomaly <- round(runif(
```

```
  nrow(climate),
```

```
  -3,
```

```
  3
```

```
),2)
```

```
head(climate)
```

```
#=====
```

```
# Heatmap 1 - Blue White Red
```

```
#=====
```

```
p1 <- ggplot(climate,
```

```
  aes(x=Year,
```

```
  y=Country,
```

```
  fill=Temperature_Anomaly))+
```

```
geom_tile()+
```

```
scale_fill_gradient2(
```

```
  low="blue",
```

```
  mid="white",
```

```
  high="red",
```

```
  midpoint=0
```


R AND PYTHON CODE 15-7-2026

```
)+  
ggtitle("Temperature Anomalies - Blue White Red")+  
theme_minimal()
```

```
print(p1)
```

```
#=====
```

Heatmap 2 - Spectral Palette

```
#=====
```

```
p2 <- ggplot(climate,  
             aes(x=Year,  
                 y=Country,  
                 fill=Temperature_Anomaly))+  
  geom_tile()+  
  scale_fill_distiller(  
    palette="Spectral"  
  )+  
  ggtitle("Temperature Anomalies - Spectral")+  
  theme_minimal()
```

```
print(p2)
```

```
#=====
```

Heatmap 3 - RdBu Palette

```
#=====
```

```
p3 <- ggplot(climate,  
             aes(x=Year,  
                 y=Country,  
                 fill=Temperature_Anomaly))+  
  geom_tile()+  
  scale_fill_distiller(  
    palette="RdBu",
```

R AND PYTHON CODE 15-7-2026

```
direction=-1

)+

ggtitle("Temperature Anomalies - RdBu")+

theme_minimal()

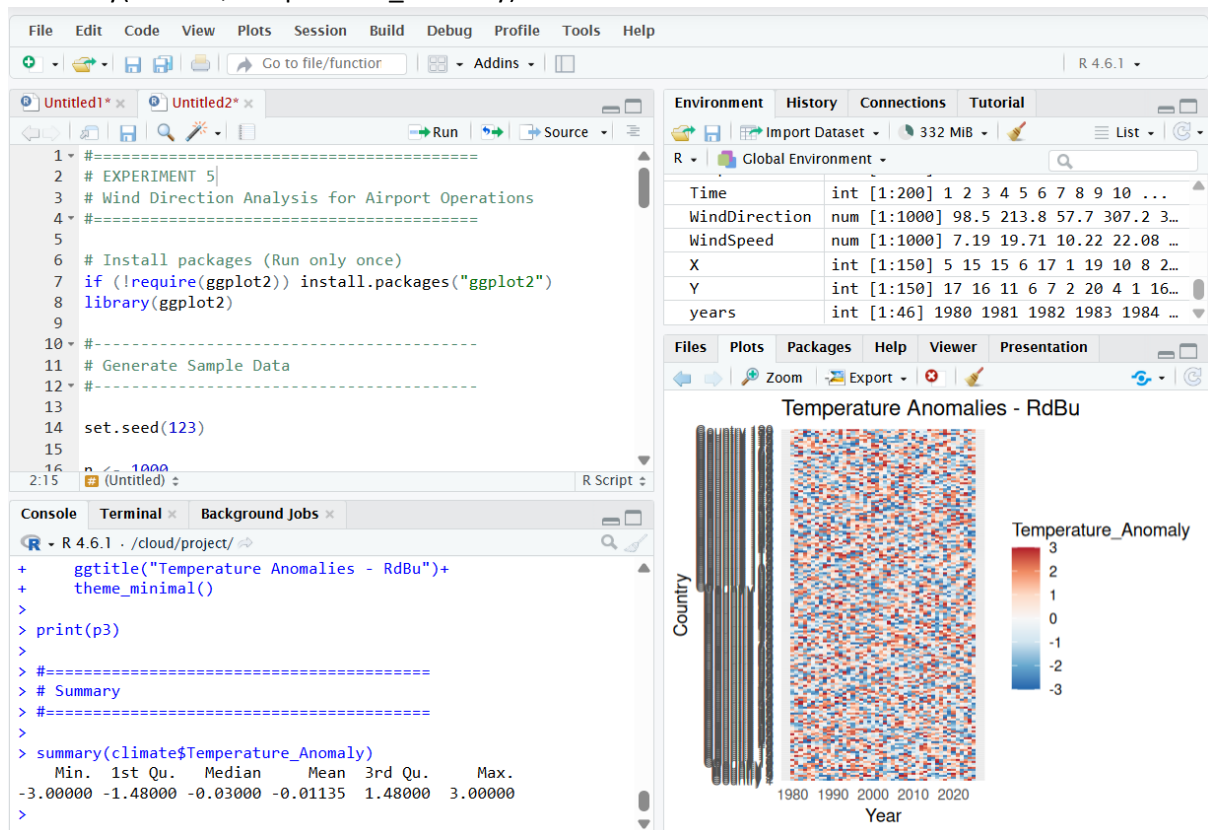
print(p3)

#=====

# Summary

#=====
```

```
summary(climate$Temperature_Anomaly)
```



```
6)

#=====

# EXPERIMENT 6

# E-Commerce Sales Analytics

#=====

# Install packages (Run only once)
```

R AND PYTHON CODE 15-7-2026

```
if (!require(ggplot2)) install.packages("ggplot2")
```

```
library(ggplot2)
```

```
#-----
```

```
# Generate Sample Data
```

```
#-----
```

```
set.seed(123)
```

```
Category <- paste("Category", 1:50)
```

```
Region <- paste("Region", 1:30)
```

```
sales <- expand.grid(
```

```
  Category = Category,
```

```
  Region = Region
```

```
)
```

```
sales$Sales <- sample(1000:100000,
```

```
  nrow(sales),
```

```
  replace = TRUE)
```

```
head(sales)
```

```
#=====
```

```
# 1. Bubble Chart
```

```
#=====
```

```
p1 <- ggplot(sales,
```

```
  aes(x = Category,
```

```
    y = Region,
```

```
    size = Sales,
```

```
    color = Category)) +
```

```
geom_point(alpha = 0.8) +
```

```
ggtitle("Bubble Chart - Sales by Category and Region") +
```

R AND PYTHON CODE 15-7-2026

```
theme_minimal() +
```

```
theme(axis.text.x = element_text(angle = 90))
```

```
print(p1)
```

```
#=====
```

```
# 2. Heatmap
```

```
#=====
```

```
p2 <- ggplot(sales,
```

```
  aes(x = Category,
```

```
    y = Region,
```

```
    fill = Sales)) +
```

```
geom_tile() +
```

```
scale_fill_gradient(low = "yellow",
```

```
  high = "red") +
```

```
ggtitle("Heatmap - Sales Volume") +
```

```
theme_minimal() +
```

```
theme(axis.text.x = element_text(angle = 90))
```

```
print(p2)
```

```
#=====
```

```
# 3. Bar Chart
```

```
#=====
```

```
total_sales <- aggregate(Sales ~ Category,
```

```
  sales,
```

```
  sum)
```

```
p3 <- ggplot(total_sales,
```

```
  aes(x = reorder(Category, Sales),
```

```
    y = Sales,
```

```
    fill = Sales)) +
```

R AND PYTHON CODE 15-7-2026

```
geom_col() +  
coord_flip() +  
scale_fill_gradient(low = "lightblue",  
                    high = "darkblue") +  
ggtitle("Total Sales by Category") +  
theme_minimal()  
  
print(p3)  
  
#=====
```

4. Scatter Plot

```
#=====
```

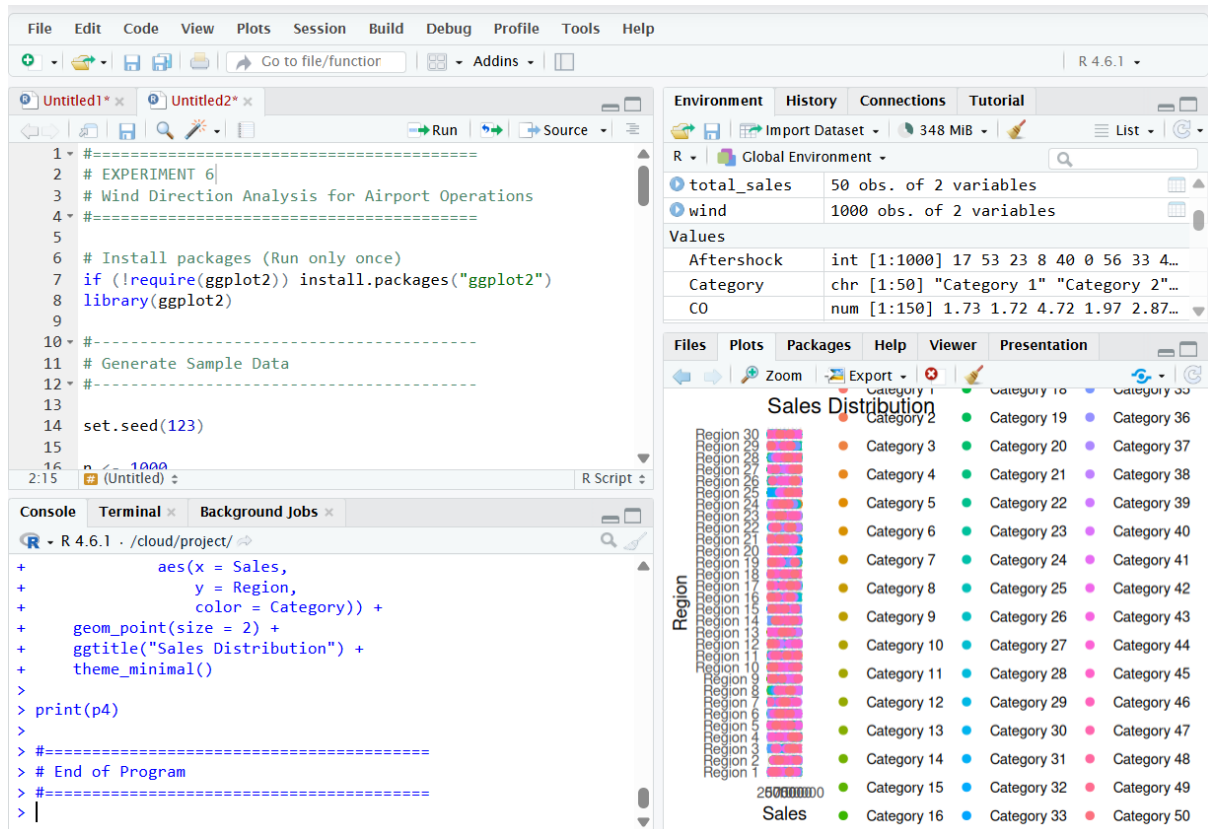


```
p4 <- ggplot(sales,  
            aes(x = Sales,  
                y = Region,  
                color = Category)) +  
geom_point(size = 2) +  
ggtitle("Sales Distribution") +  
theme_minimal()  
  
print(p4)  
  
#=====
```

End of Program

```
#=====
```

R AND PYTHON CODE 15-7-2026



7)

```
#=====
```

```
# EXPERIMENT 7
```

```
# Satellite Image Vegetation Analysis
```

```
#=====
```

```
# Install packages (Run only once)
```

```
if (!require(ggplot2)) install.packages("ggplot2")
```

```
library(ggplot2)
```

```
#-----
```

```
# Generate Sample NDVI Data
```

```
#-----
```

```
set.seed(123)
```

```
n <- 1000
```

R AND PYTHON CODE 15-7-2026

```
X <- sample(1:100, n, replace = TRUE)
```

```
Y <- sample(1:100, n, replace = TRUE)
```

```
NDVI <- round(runif(n, -1, 1), 2)
```

```
vegetation <- data.frame(
```

```
  X,
```

```
  Y,
```

```
  NDVI
```

```
)
```

```
head(vegetation)
```

```
#=====
```

```
# 1. Sequential Green Palette
```

```
#=====
```

```
p1 <- ggplot(vegetation,
```

```
  aes(x = X,
```

```
    y = Y,
```

```
    color = NDVI)) +
```

```
geom_point(size = 3) +
```

```
scale_color_gradient(
```

```
  low = "yellow",
```

```
  high = "darkgreen"
```

```
) +
```

```
ggtitle("Sequential Palette - Vegetation Health") +
```

```
theme_minimal()
```

```
print(p1)
```

```
#=====
```

```
# 2. Diverging Palette
```

```
#=====
```

R AND PYTHON CODE 15-7-2026

```
p2 <- ggplot(vegetation,  
  aes(x = X,  
    y = Y,  
    color = NDVI)) +  
geom_point(size = 3) +  
scale_color_gradient2(  
  low = "blue",  
  mid = "yellow",  
  high = "darkgreen",  
  midpoint = 0  
) +  
ggtitle("Diverging Palette - NDVI") +  
theme_minimal()
```

```
print(p2)
```

```
#=====
```

```
# 3. NDVI Histogram
```

```
#=====
```

```
p3 <- ggplot(vegetation,  
  aes(x = NDVI)) +  
geom_histogram(  
  bins = 25,  
  fill = "forestgreen",  
  color = "black"  
) +  
ggtitle("NDVI Distribution") +  
theme_minimal()
```

```
print(p3)
```

```
#=====
```


R AND PYTHON CODE 15-7-2026

4. NDVI Heatmap

```
#=====
```

```
p4 <- ggplot(vegetation,  
  aes(x = X,  
    y = Y,  
    fill = NDVI)) +  
  geom_tile() +  
  scale_fill_gradient2(  
    low = "blue",  
    mid = "yellow",  
    high = "darkgreen",  
    midpoint = 0  
  ) +  
  ggtitle("NDVI Heatmap") +  
  theme_minimal()
```

```
print(p4)
```

```
#=====
```

```
# NDVI Classification
```

```
#=====
```

```
vegetation$Class <- ifelse(  
  vegetation$NDVI < 0,  
  "Water",  
  ifelse(  
    vegetation$NDVI < 0.3,  
    "Barren Land",  
    "Healthy Vegetation"  
  )  
)
```

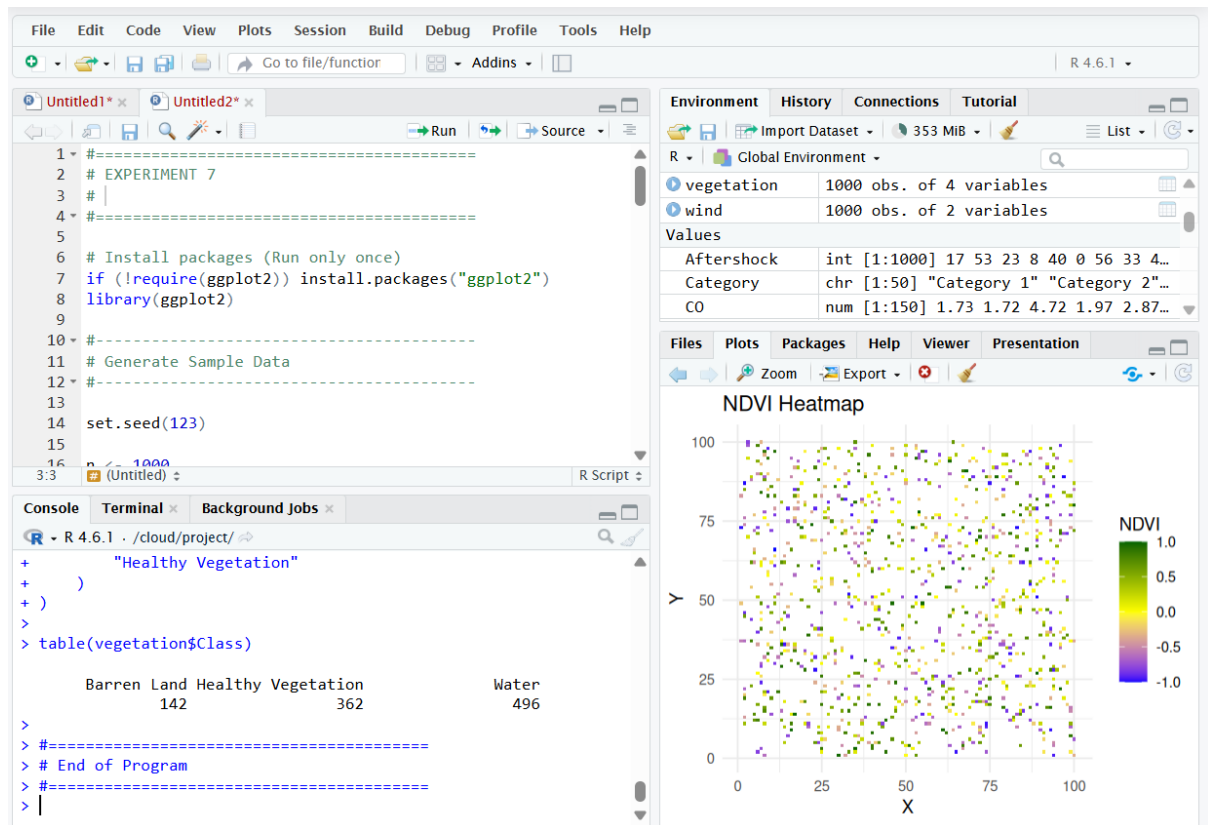
```
table(vegetation$Class)
```

R AND PYTHON CODE 15-7-2026

```
#=====
```

```
# End of Program
```

```
#=====
```



8)

```
#=====
```

```
# EXPERIMENT 8
```

```
# Cryptocurrency Market Volatility
```

```
#=====
```

```
# Install package (Run only once)
```

```
if (!require(ggplot2)) install.packages("ggplot2")
```

```
library(ggplot2)
```

```
#-----
```

```
# Generate Sample Cryptocurrency Data
```

```
#-----
```

```
set.seed(123)
```

R AND PYTHON CODE 15-7-2026

```
Days <- 1:365
```

```
Bitcoin <- round(runif(365, 30000, 110000),2)
```

```
Ethereum <- round(runif(365, 1500, 7000),2)
```

```
Solana <- round(runif(365, 20, 350),2)
```

```
Cardano <- round(runif(365, 0.2, 3),2)
```

```
crypto <- data.frame(
```

```
  Days,
```

```
  Bitcoin,
```

```
  Ethereum,
```

```
  Solana,
```

```
  Cardano
```

```
)
```

```
head(crypto)
```

```
#=====
```

```
# Convert to Long Format
```

```
#=====
```

```
coins <- data.frame(
```

```
  Day = rep(Days,4),
```

```
  Price = c(Bitcoin, Ethereum, Solana, Cardano),
```

```
  Currency = rep(c("Bitcoin",
```

```
    "Ethereum",
```

```
    "Solana",
```

```
    "Cardano"),
```

```
  each = 365)
```

```
)
```

```
#=====
```

```
# 1. Cartesian Line Plot
```

R AND PYTHON CODE 15-7-2026

```
#=====
```

```
p1 <- ggplot(coins,
             aes(x = Day,
                 y = Price,
                 color = Currency)) +
  geom_line(size = 1) +
  ggtitle("Cryptocurrency Prices (Linear Scale)") +
  xlab("Day") +
  ylab("Price (USD)") +
  theme_minimal()
```

```
print(p1)
```

```
#=====
```

```
# 2. Logarithmic Scale Plot
```

```
#=====
```

```
p2 <- ggplot(coins,
             aes(x = Day,
                 y = Price,
                 color = Currency)) +
  geom_line(size = 1) +
  scale_y_log10() +
  ggtitle("Cryptocurrency Prices (Log Scale)") +
  xlab("Day") +
  ylab("Log Price") +
  theme_minimal()
```

```
print(p2)
```

```
#=====
```

```
# 3. Box Plot
```

```
#=====
```

R AND PYTHON CODE 15-7-2026

```
p3 <- ggplot(coins,  
             aes(x = Currency,  
                 y = Price,  
                 fill = Currency)) +  
  geom_boxplot() +  
  ggtitle("Price Distribution of Cryptocurrencies") +  
  theme_minimal()  
  
print(p3)
```

```
#=====
```

4. Histogram

```
#=====
```

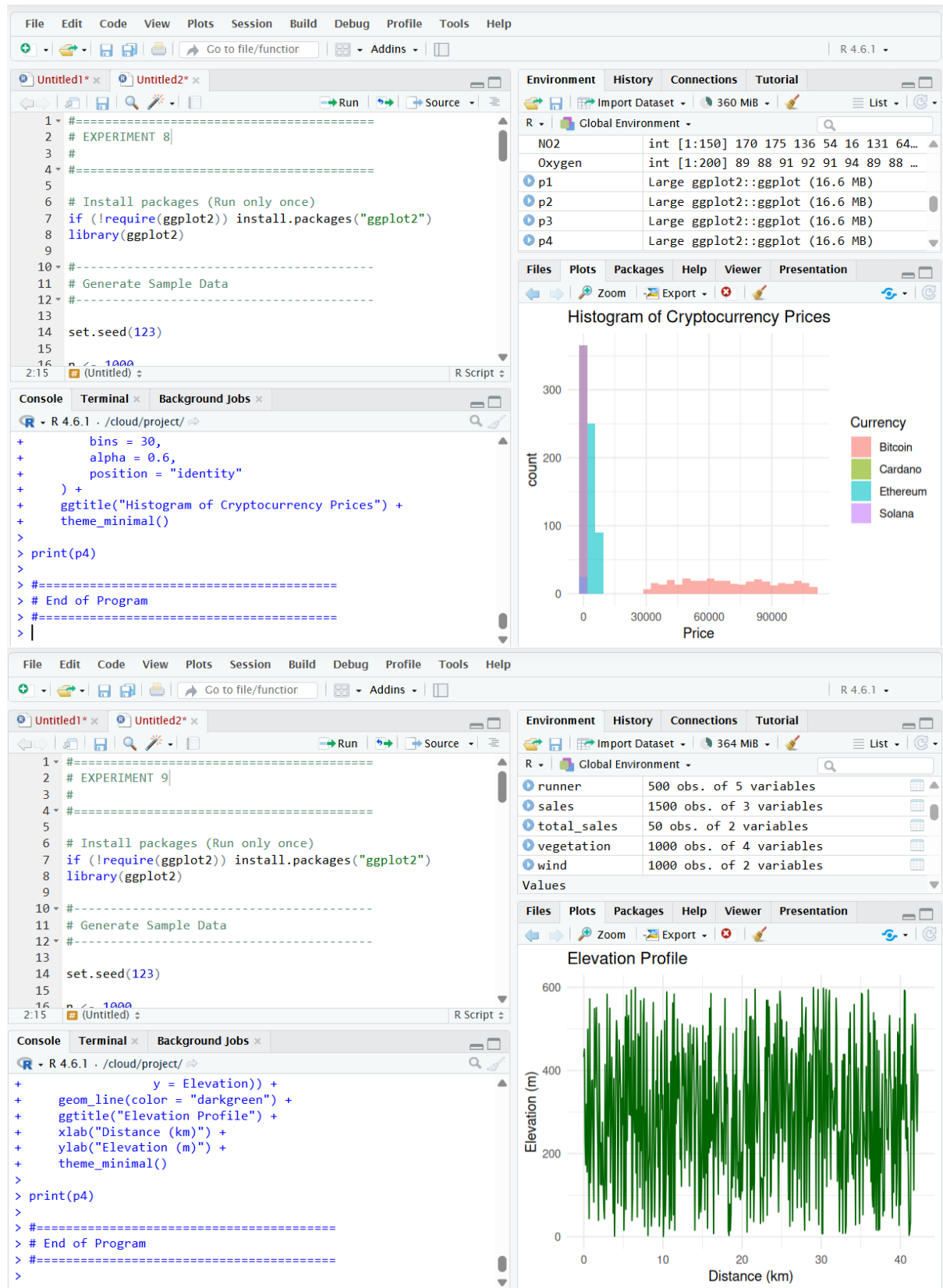
```
p4 <- ggplot(coins,  
             aes(x = Price,  
                 fill = Currency)) +  
  geom_histogram(  
    bins = 30,  
    alpha = 0.6,  
    position = "identity"  
  ) +  
  ggtitle("Histogram of Cryptocurrency Prices") +  
  theme_minimal()  
  
print(p4)
```

```
#=====
```

End of Program

R AND PYTHON CODE 15-7-2026

#=====



9)

#=====

EXPERIMENT 9

Marathon Runner Performance Analysis

R AND PYTHON CODE 15-7-2026

```
#=====
```

```
# Install package (Run only once)
```

```
if (!require(ggplot2)) install.packages("ggplot2")
```

```
library(ggplot2)
```

```
#-----
```

```
# Generate Sample Data
```

```
#-----
```

```
set.seed(123)
```

```
n <- 500
```

```
Distance <- seq(0, 42.2, length.out = n)
```

```
Speed <- round(runif(n, 6, 18), 2)
```

```
HeartRate <- sample(90:190, n, replace = TRUE)
```

```
Elevation <- round(runif(n, 0, 600), 2)
```

```
Direction <- runif(n, 0, 360)
```

```
runner <- data.frame(
```

```
  Distance,
```

```
  Speed,
```

```
  HeartRate,
```

```
  Elevation,
```

```
  Direction
```

```
)
```

```
head(runner)
```

```
#=====
```

```
# 1. Cartesian Plot
```

```
#=====
```

R AND PYTHON CODE 15-7-2026

```
p1 <- ggplot(runner,
             aes(x = Distance,
                 y = Speed,
                 color = Speed)) +
  geom_line(size = 1) +
  scale_color_gradient(
    low = "blue",
    high = "red"
  ) +
  ggtitle("Runner Speed vs Distance") +
  xlab("Distance (km)") +
  ylab("Speed (km/h)") +
  theme_minimal()
```

```
print(p1)
```

```
#=====
```

```
# 2. Polar Coordinate Plot
```

```
#=====
```

```
p2 <- ggplot(runner,
             aes(x = factor(round(Direction/30)*30),
                 fill = Speed)) +
  geom_bar() +
  coord_polar() +
  ggtitle("Running Direction (Polar Plot)") +
  theme_minimal()
```

```
print(p2)
```

```
#=====
```

```
# 3. Heart Rate Plot
```

```
#=====
```


R AND PYTHON CODE 15-7-2026

```
p3 <- ggplot(runner,
             aes(x = Distance,
                 y = HeartRate,
                 color = HeartRate)) +
  geom_line(size = 1) +
  scale_color_gradient(
    low = "green",
    high = "red"
  ) +
  ggtitle("Heart Rate During Marathon") +
  theme_minimal()
```

```
print(p3)
```

```
#=====
```

```
# 4. Elevation Plot
```

```
#=====
```

```
p4 <- ggplot(runner,
             aes(x = Distance,
                 y = Elevation)) +
  geom_line(color = "darkgreen") +
  ggtitle("Elevation Profile") +
  xlab("Distance (km)") +
  ylab("Elevation (m)") +
  theme_minimal()
```

```
print(p4)
```

```
#=====
```

```
# End of Program
```

```
#=====
```

R AND PYTHON CODE 15-7-2026

```
1 #=====
2 # EXPERIMENT 10|
3 #
4 #=====
5
6 # Install packages (Run only once)
7 if (!require(ggplot2)) install.packages("ggplot2")
8 library(ggplot2)
9
10 #-----
11 # Generate Sample Data
12 #-----
13
14 set.seed(123)
15
16 n <- 1000
2:16 (Untitled) R Script
```

```
+ low = "white",
+ high = "black"
+ ) +
+ ggtitle("Color-Blind Friendly Dashboard") +
+ theme_minimal() +
+ theme(axis.text.x = element_text(angle = 90))
> print(p5)
>
> #=====
> # End of Program
> #=====
>
```

Environment History Connections Tutorial

R - Global Environment

vaccination	360 obs. of 4 variables
vegetation	1000 obs. of 4 variables
wind	1000 obs. of 2 variables

Values

Aftershock	int [1:1000] 17 53 23 8 40 0 56 33 4...
AgeGroup	chr [1:4] "0-18" "19-40" "41-60" "60..."

Files Plots Packages Help Viewer Presentation

Zoom Export

Color-Blind Friendly Dashboard

AgeGroup

District

```
1 #=====
2 # EXPERIMENT 9|
3 #
4 #=====
5
6 # Install packages (Run only once)
7 if (!require(ggplot2)) install.packages("ggplot2")
8 library(ggplot2)
9
10 #-----
11 # Generate Sample Data
12 #-----
13
14 set.seed(123)
15
16 n <- 1000
2:15 (Untitled) R Script
```

```
+ y = Elevation)) +
+ geom_line(color = "darkgreen") +
+ ggtitle("Elevation Profile") +
+ xlab("Distance (km)") +
+ ylab("Elevation (m)") +
+ theme_minimal()
> print(p4)
>
> #=====
> # End of Program
> #=====
>
```

Environment History Connections Tutorial

R - Global Environment

runner	500 obs. of 5 variables
sales	1500 obs. of 3 variables
total_sales	50 obs. of 2 variables
vegetation	1000 obs. of 4 variables
wind	1000 obs. of 2 variables

Values

Files Plots Packages Help Viewer Presentation

Zoom Export

Elevation Profile

Elevation (m)

Distance (km)

10)

#=====

EXPERIMENT 10

COVID-19 Vaccination Campaign Dashboard

#=====

R AND PYTHON CODE 15-7-2026

```
# Install packages (Run only once)

if (!require(ggplot2)) install.packages("ggplot2")

library(ggplot2)

#-----

# Generate Sample Data

#-----

set.seed(123)

District <- paste("District", 1:30)
AgeGroup <- c("0-18", "19-40", "41-60", "60+")
Vaccine <- c("Covishield", "Covaxin", "Sputnik")

vaccination <- expand.grid(
  District = District,
  AgeGroup = AgeGroup,
  Vaccine = Vaccine
)

vaccination$VaccinationRate <- sample(40:100,
                                     nrow(vaccination),
                                     replace = TRUE)

head(vaccination)

#=====

# 1. Heatmap for Policymakers

#=====

p1 <- ggplot(vaccination,
             aes(x = District,
                 y = AgeGroup,
```

R AND PYTHON CODE 15-7-2026

```
      fill = VaccinationRate)) +  
  
geom_tile() +  
  
scale_fill_gradient(  
  low = "yellow",  
  high = "darkgreen"  
) +  
  
ggtitle("COVID-19 Vaccination Rates (Policy Dashboard)") +  
  
xlab("District") +  
  
ylab("Age Group") +  
  
theme_minimal() +  
  
theme(axis.text.x = element_text(angle = 90))  
  
  
print(p1)  
  
  
#=====
```

2. Public Dashboard - Bar Chart

```
#=====
```



```
district_total <- aggregate(  
  VaccinationRate ~ District,  
  vaccination,  
  mean  
)  
  
  
p2 <- ggplot(district_total,  
  aes(x = reorder(District, VaccinationRate),  
    y = VaccinationRate,  
    fill = VaccinationRate)) +  
  
geom_col() +  
  
coord_flip() +  
  
scale_fill_gradient(  
  low = "lightblue",  
  high = "blue"  
) +
```

R AND PYTHON CODE 15-7-2026

```
ggtitle("Average Vaccination Rate by District") +  
xlab("District") +  
ylab("Average Vaccination Rate (%)") +  
theme_minimal()
```

```
print(p2)
```

```
#=====
```

3. Vaccine-wise Box Plot

```
#=====
```

```
p3 <- ggplot(vaccination,  
             aes(x = Vaccine,  
                 y = VaccinationRate,  
                 fill = Vaccine)) +  
geom_boxplot() +  
ggtitle("Vaccination Rate by Vaccine Type") +  
theme_minimal()
```

```
print(p3)
```

```
#=====
```

4. Age Group Comparison

```
#=====
```

```
age_total <- aggregate(  
  VaccinationRate ~ AgeGroup,  
  vaccination,  
  mean  
)
```

```
p4 <- ggplot(age_total,  
             aes(x = AgeGroup,  
                 y = VaccinationRate,
```

R AND PYTHON CODE 15-7-2026

```
      fill = AgeGroup)) +  
geom_bar(stat = "identity") +  
ggtitle("Average Vaccination Rate by Age Group") +  
xlab("Age Group") +  
ylab("Average Vaccination Rate (%)") +  
theme_minimal()
```

```
print(p4)
```

```
#=====
```

```
# 5. Accessibility Version
```

```
#=====
```

```
p5 <- ggplot(vaccination,  
      aes(x = District,  
          y = AgeGroup,  
          fill = VaccinationRate)) +  
geom_tile(color = "black") +  
scale_fill_gradient(  
  low = "white",  
  high = "black"  
) +  
ggtitle("Color-Blind Friendly Dashboard") +  
theme_minimal() +  
theme(axis.text.x = element_text(angle = 90))
```

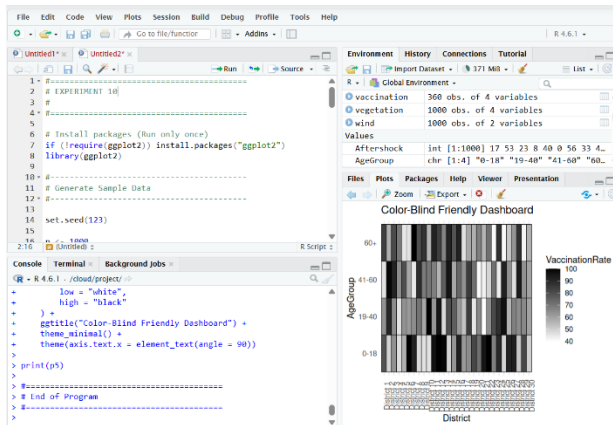
```
print(p5)
```

```
#=====
```

```
# End of Program
```

```
#=====
```

R AND PYTHON CODE 15-7-2026



Python codes

1. Smart City Air Quality Monitoring

import matplotlib.pyplot as plt

Sample data

location = [1, 2, 3, 4, 5]

pm25 = [30, 60, 90, 50, 80]

level = ["Low", "Moderate", "High", "Moderate", "High"]

1. Sequential Palette

plt.scatter(location, pm25, c=pm25, cmap="Blues", s=100) plt.title("Sequential Palette")

plt.xlabel("Location") plt.ylabel("PM2.5") plt.colorbar(label="PM2.5") plt.show()

2. Diverging Palette

plt.scatter(location, pm25, c=pm25, cmap="coolwarm", s=100) plt.title("Diverging Palette")

plt.xlabel("Location") plt.ylabel("PM2.5") plt.colorbar(label="PM2.5") plt.show()

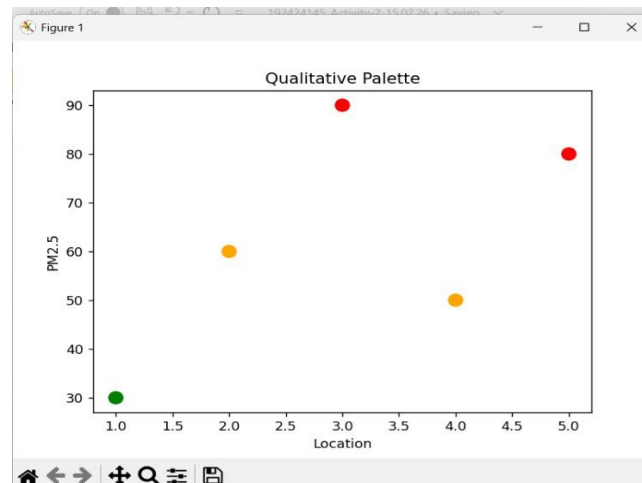
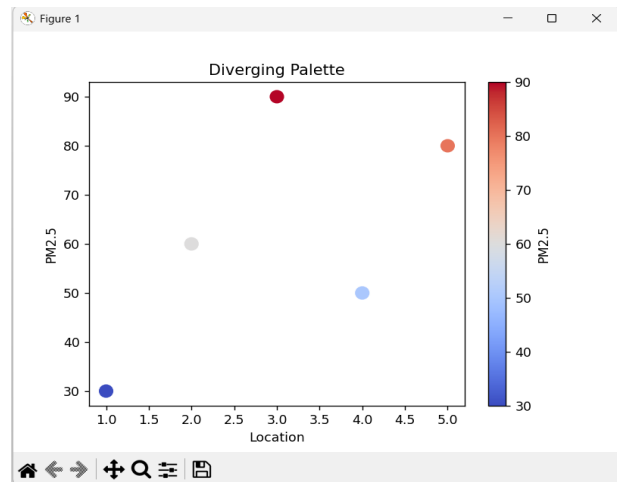
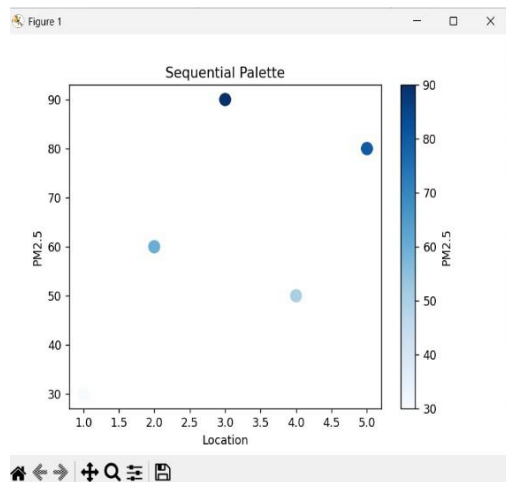
3. Qualitative Palette

colors = ["green", "orange", "red", "orange", "red"] plt.scatter(location, pm25, c=colors, s=100)

plt.title("Qualitative Palette")

R AND PYTHON CODE 15-7-2026

```
plt.xlabel("Location") plt.ylabel("PM2.5") plt.show()
```



R AND PYTHON CODE 15-7-2026

2. Earthquake Risk Analysis

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
magnitude = [0.5, 1, 2, 3, 5, 7, 9]
```

```
depth = [20, 40, 80, 150, 250, 500, 700]
```

```
# Linear Scale
```

```
plt.scatter(magnitude, depth, color="blue") plt.title("Earthquake Risk Analysis (Linear Scale)")
```

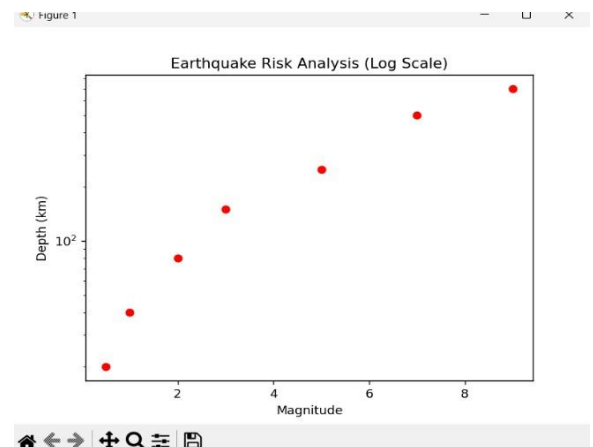
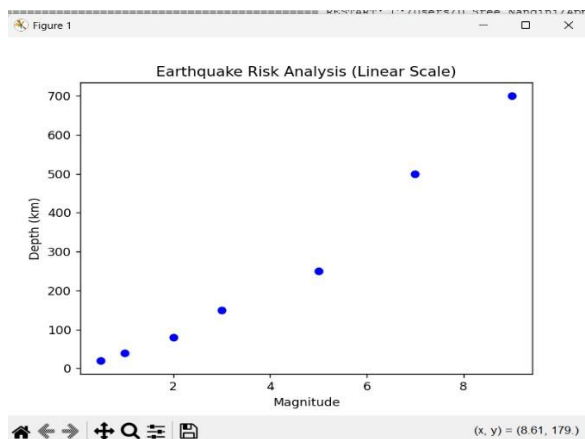
```
plt.xlabel("Magnitude")
```

```
plt.ylabel("Depth (km)") plt.show()
```

```
# Logarithmic Scale plt.scatter(magnitude, depth, color="red") plt.yscale("log")
```

```
plt.title("Earthquake Risk Analysis (Log Scale)") plt.xlabel("Magnitude")
```

```
plt.ylabel("Depth (km)") plt.show()
```



R AND PYTHON CODE 15-7-2026

3. Wind Direction Analysis for Airport Operations

Cartesian Plot

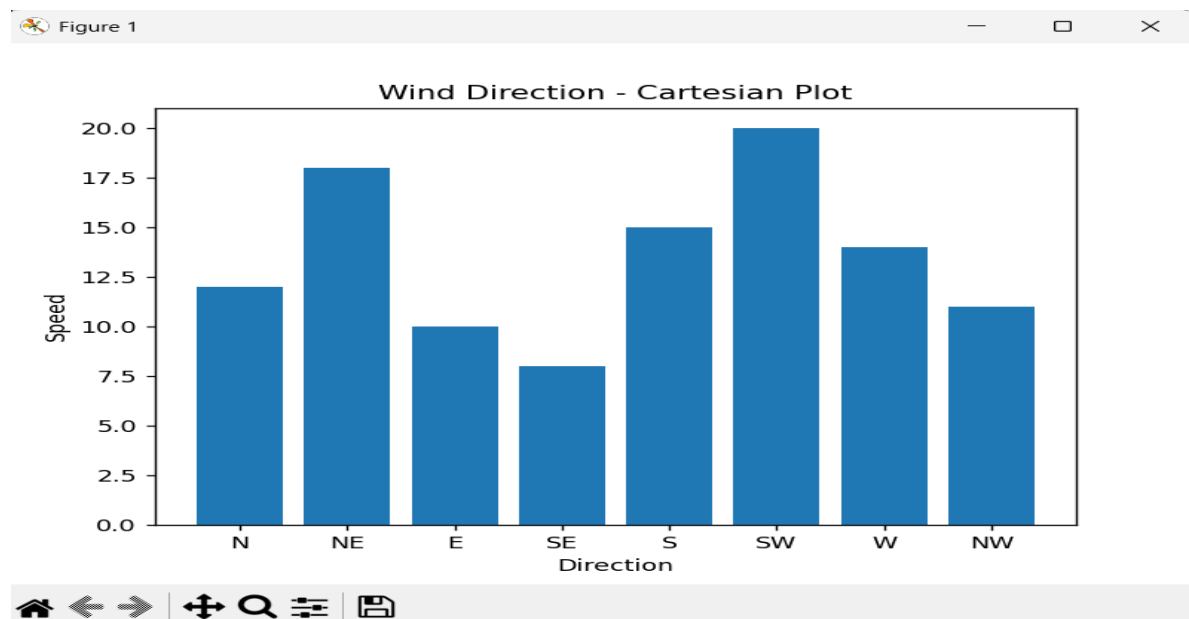
```
import matplotlib.pyplot as plt
```

```
direction = ["N", "NE", "E", "SE", "S", "SW", "W", "NW"] speed = [12, 18, 10, 8, 15, 20, 14, 11]
```

```
plt.bar(direction, speed)
```

```
plt.title("Wind Direction - Cartesian Plot") plt.xlabel("Direction")
```

```
plt.ylabel("Speed") plt.show()
```



R AND PYTHON CODE 15-7-2026

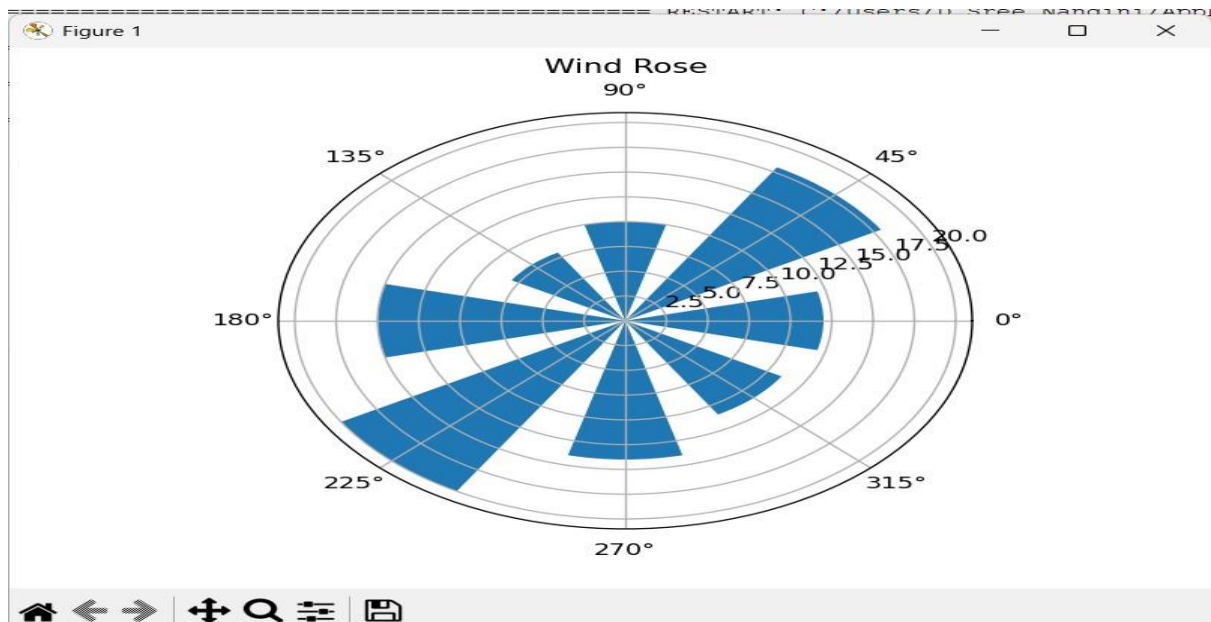
Polar Plot (Wind Rose) import matplotlib.pyplot as plt import numpy as np

```
speed = [12, 18, 10, 8, 15, 20, 14, 11]
```

```
theta = np.linspace(0, 2*np.pi, len(speed), endpoint=False)
```

```
ax = plt.subplot(111, polar=True) ax.bar(theta, speed, width=0.5)
```

```
plt.title("Wind Rose") plt.show()
```



R AND PYTHON CODE 15-7-2026

4. Hospital Patient Monitoring Dashboard

```
import matplotlib.pyplot as plt
```

```
parameters = ["Heart Rate", "Oxygen", "BP", "Respiration", "Temperature"] values = [130, 88, 140, 28, 39]
```

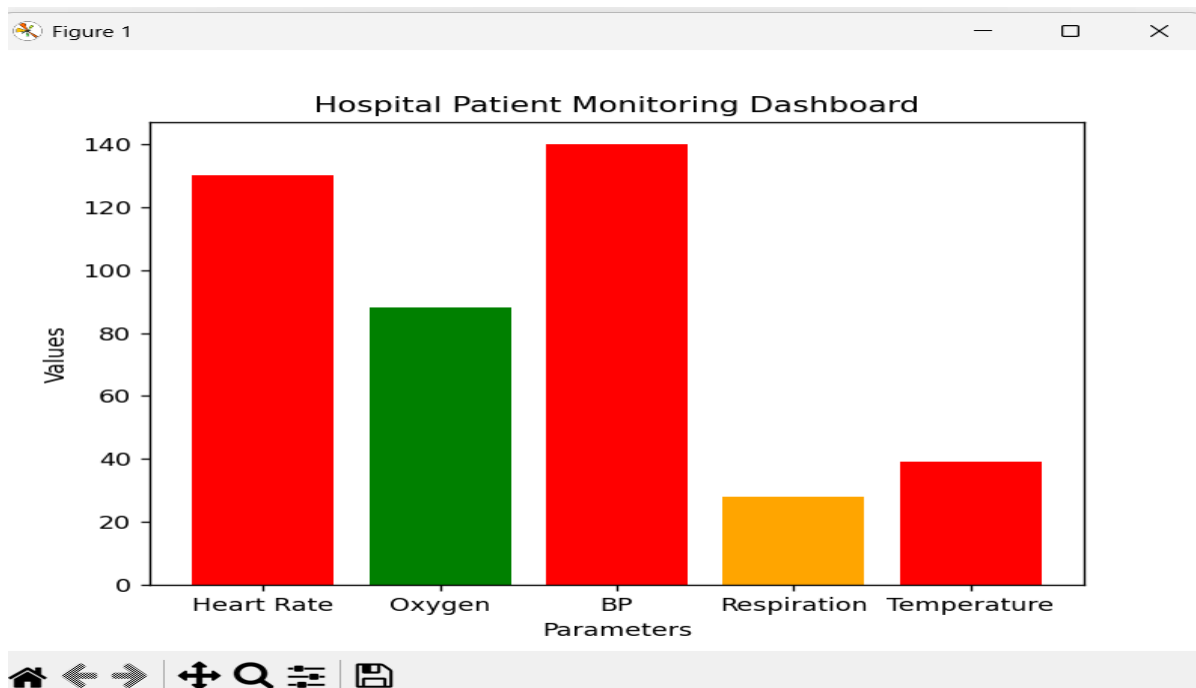
```
colors = ["red", "green", "red", "orange", "red"]
```

```
plt.bar(parameters, values, color=colors)
```

```
plt.title("Hospital Patient Monitoring Dashboard") plt.xlabel("Parameters")
```

```
plt.ylabel("Values")
```

```
plt.show()
```



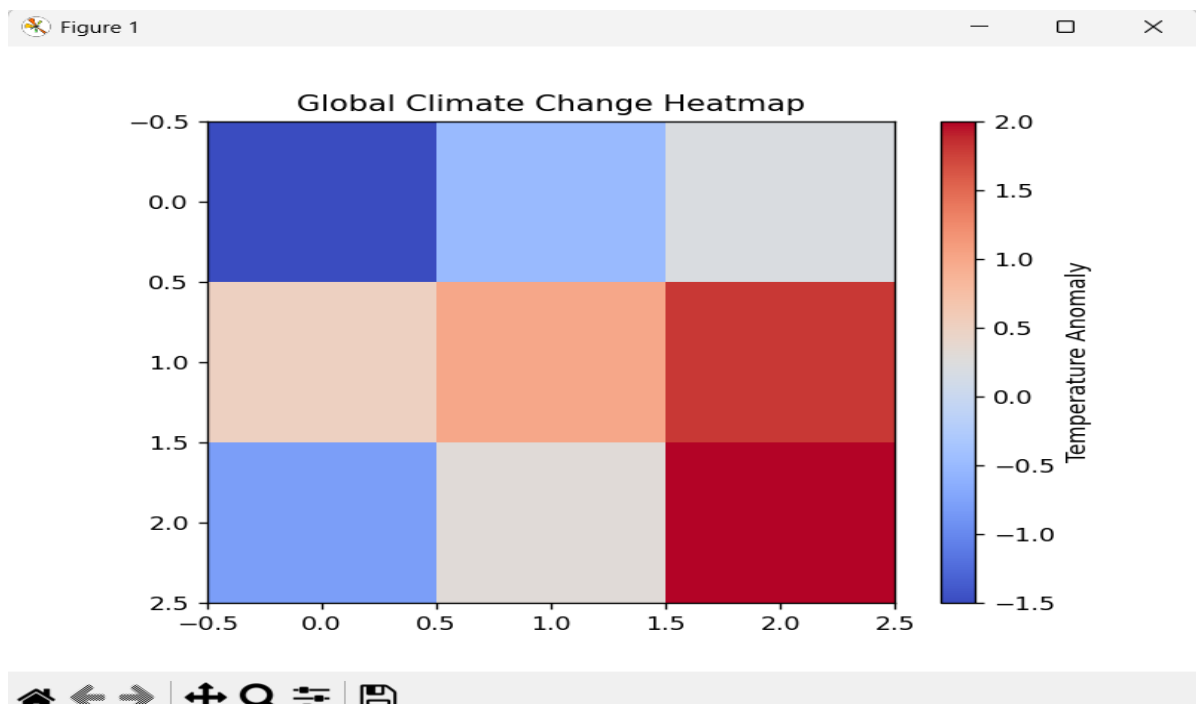
R AND PYTHON CODE 15-7-2026

5. **Global Climate Change Study** import matplotlib.pyplot as plt import numpy as np

```
data = np.array([  
[-1.5, -0.5, 0.2],  
[0.5, 1.0, 1.8],  
[-0.8, 0.3, 2.0]  
])
```

Heatmap

```
plt.imshow(data, cmap="coolwarm") plt.colorbar(label="Temperature Anomaly") plt.title("Global Climate  
Change Heatmap") plt.show()
```



R AND PYTHON CODE 15-7-2026

6.E-Commerce Sales Analytics

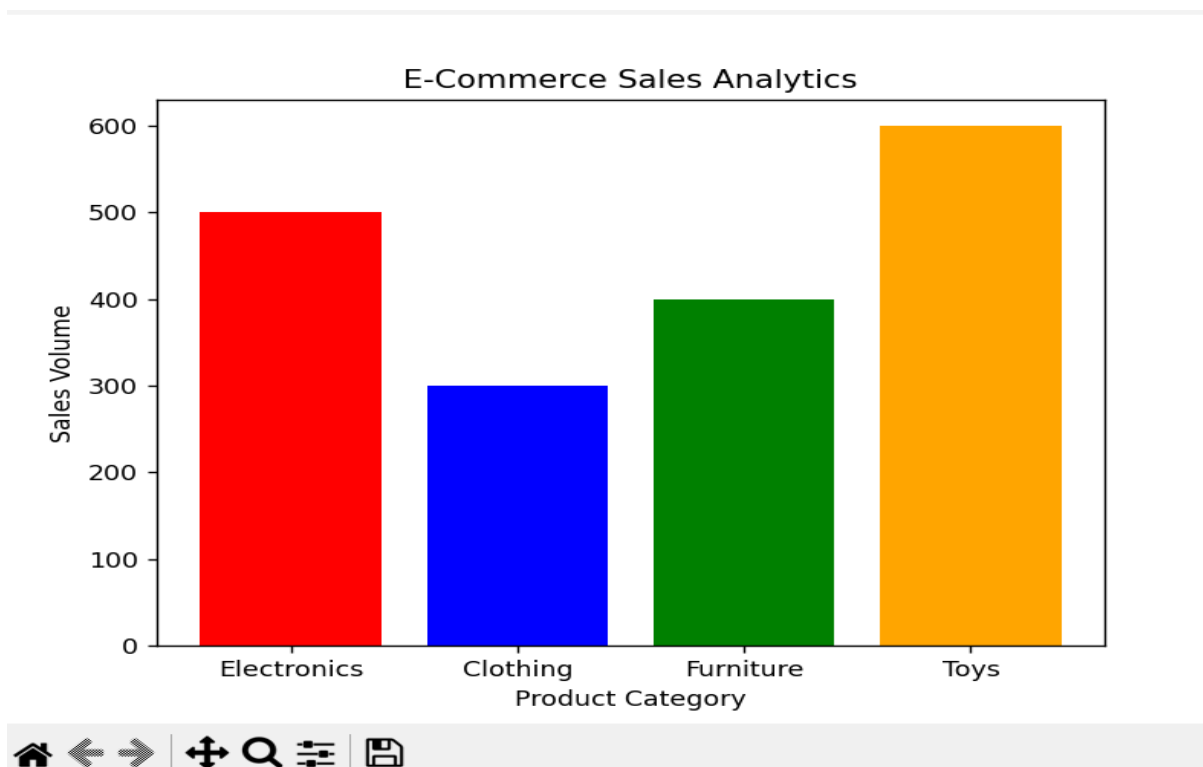
```
import matplotlib.pyplot as plt
```

```
category = ["Electronics", "Clothing", "Furniture", "Toys"] sales = [500, 300, 400, 600]
```

```
plt.bar(category, sales, color=["red", "blue", "green", "orange"])
```

```
plt.title("E-Commerce Sales Analytics") plt.xlabel("Product Category") plt.ylabel("Sales Volume")
```

```
plt.show()
```



R AND PYTHON CODE 15-7-2026

7.Satellite Image Vegetation Analysis

```
import matplotlib.pyplot as plt
```

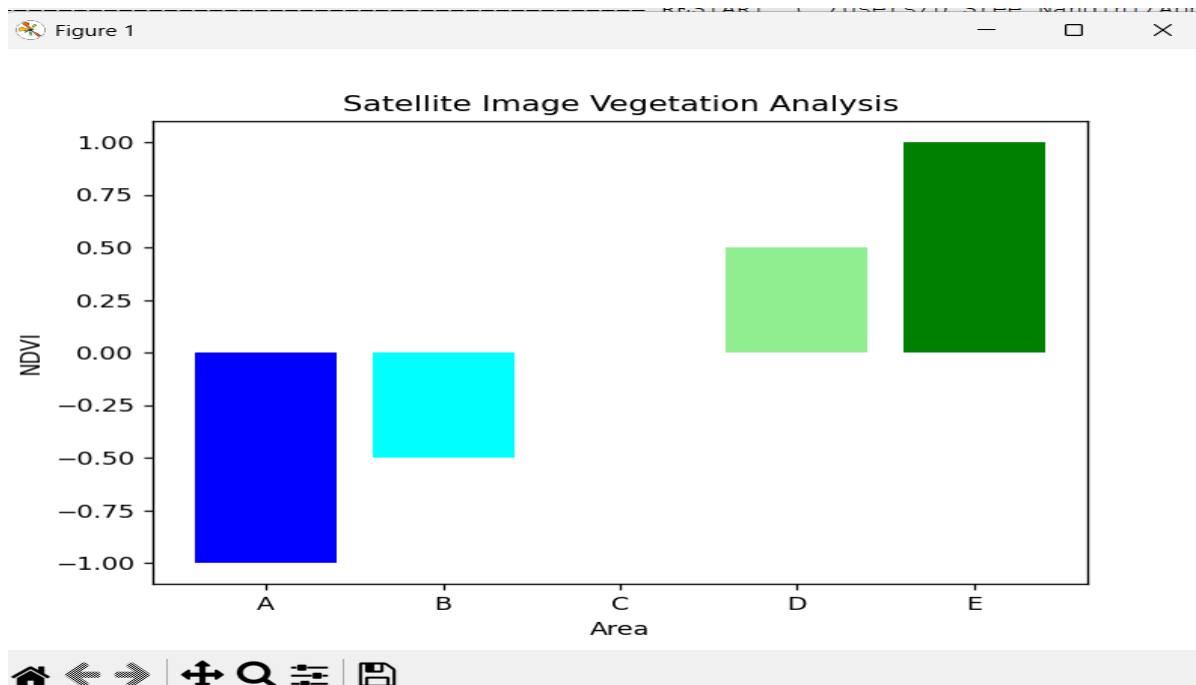
```
area = ["A", "B", "C", "D", "E"] ndvi = [-1, -0.5, 0, 0.5, 1]
```

```
plt.bar(area, ndvi, color=["blue", "cyan", "yellow", "lightgreen", "green"])
```

```
plt.title("Satellite Image Vegetation Analysis") plt.xlabel("Area")
```

```
plt.ylabel("NDVI")
```

```
plt.show()
```



R AND PYTHON CODE 15-7-2026

8. Cryptocurrency Market Volatility

```
import matplotlib.pyplot as plt
```

```
day = [1, 2, 3, 4, 5]
```

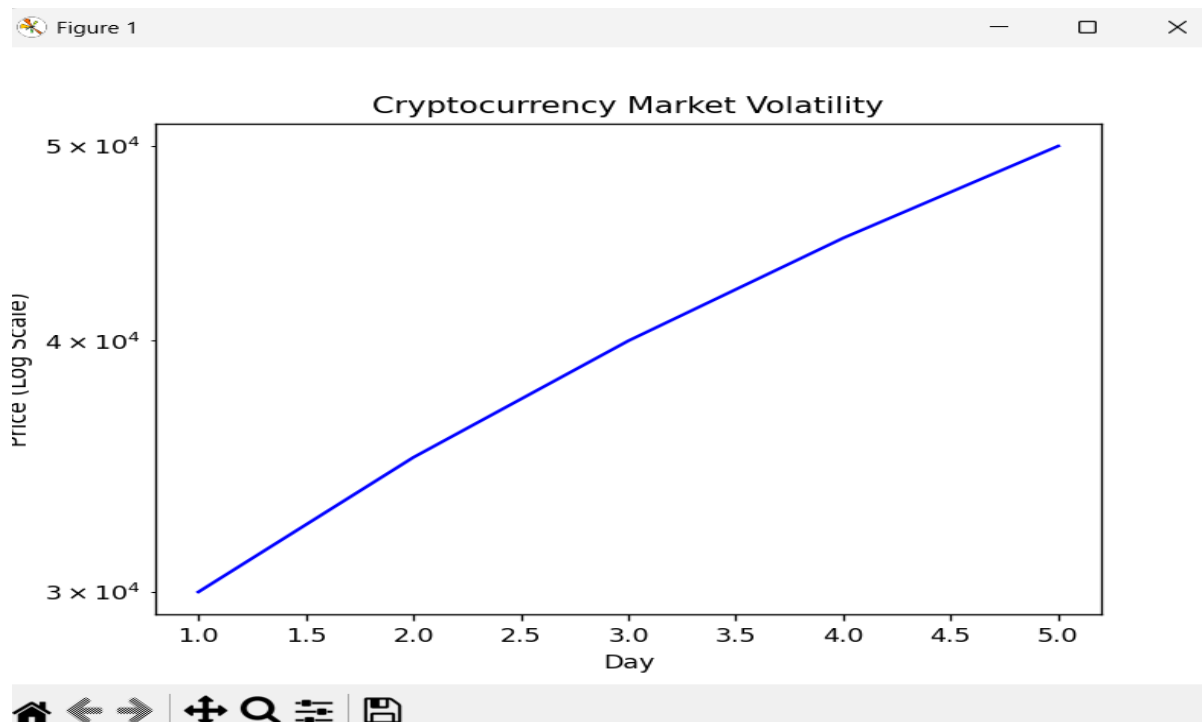
```
bitcoin = [30000, 35000, 40000, 45000, 50000]
```

```
plt.plot(day, bitcoin, color="blue") plt.yscale("log")
```

```
plt.title("Cryptocurrency Market Volatility") plt.xlabel("Day")
```

```
plt.ylabel("Price (Log Scale)")
```

```
plt.show()
```



R AND PYTHON CODE 15-7-2026

9.Marathon Runner Performance Analysis

Cartesian Plot

```
import matplotlib.pyplot as plt
```

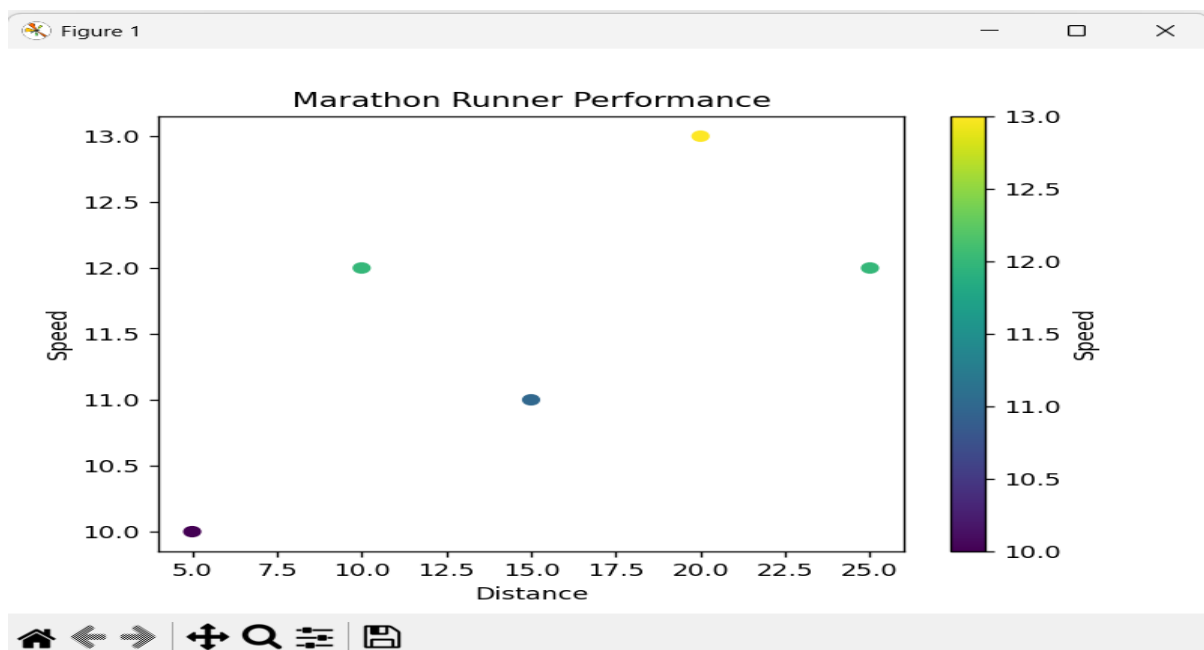
```
distance = [5, 10, 15, 20, 25]
```

```
speed = [10, 12, 11, 13, 12]
```

```
plt.scatter(distance, speed, c=speed, cmap="viridis") plt.colorbar(label="Speed")
```

```
plt.title("Marathon Runner Performance") plt.xlabel("Distance")
```

```
plt.ylabel("Speed") plt.show()
```



R AND PYTHON CODE 15-7-2026

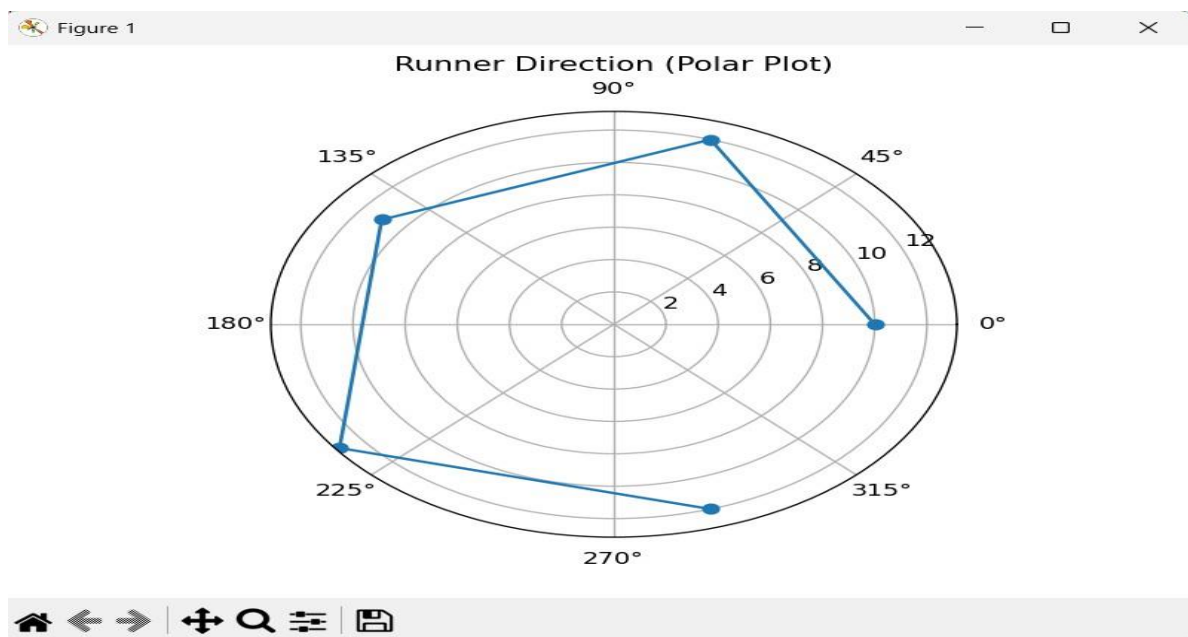
Polar Plot

```
import matplotlib.pyplot as plt import numpy as np
```

```
speed = [10, 12, 11, 13, 12]
```

```
angle = np.linspace(0, 2*np.pi, len(speed), endpoint=False)
```

```
plt.polar(angle, speed, 'o-') plt.title("Runner Direction (Polar Plot)") plt.show()
```



R AND PYTHON CODE 15-7-2026

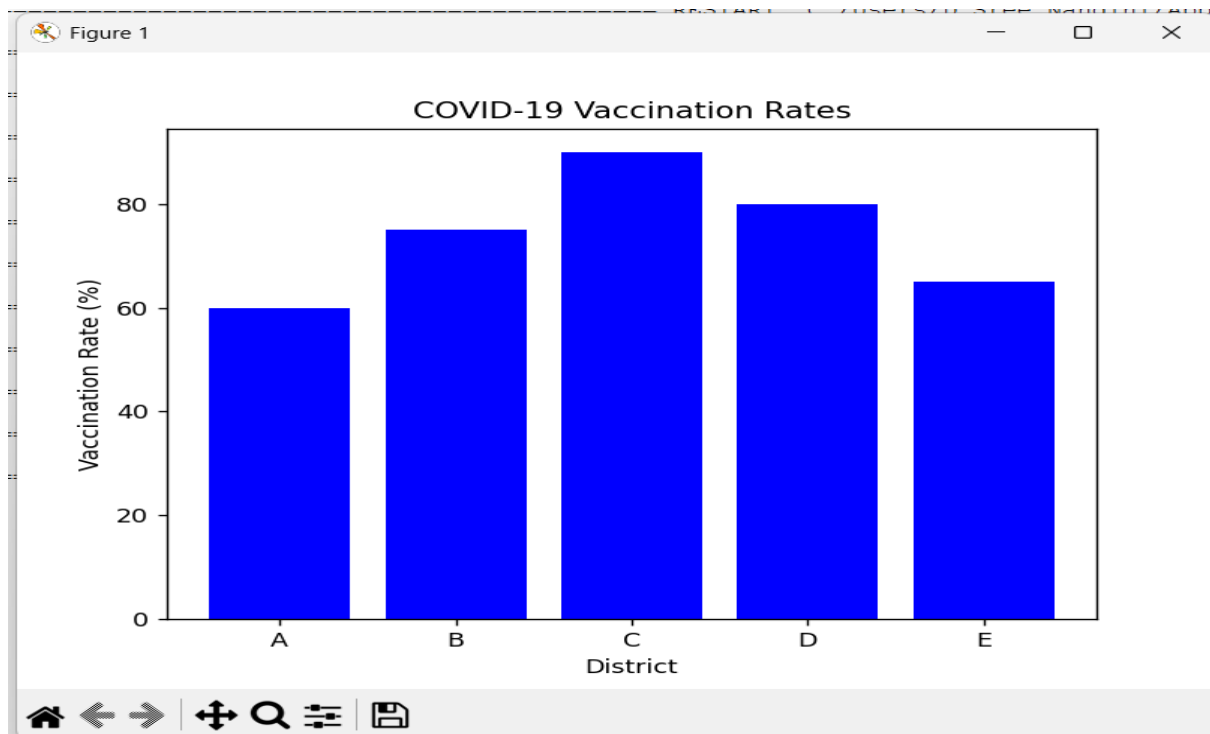
10. COVID-19 Vaccination Campaign Dashboard

Version 1 – Blue Color Palette (For Policymakers)

```
import matplotlib.pyplot as plt
```

```
district = ["A", "B", "C", "D", "E"] rate = [60, 75, 90, 80, 65]
```

```
plt.bar(district, rate, color="blue") plt.title("COVID-19 Vaccination Rates") plt.xlabel("District")  
plt.ylabel("Vaccination Rate (%)") plt.show()
```



R AND PYTHON CODE CLASS ACTIVITY

-15/7/26

Version 2 – Green Color Palette (For General Public)

```
import matplotlib.pyplot as plt
```

```
district = ["A", "B", "C", "D", "E"] rate = [60, 75, 90, 80, 65]
```

```
plt.bar(district, rate, color="green") plt.title("COVID-19 Vaccination Rates") plt.xlabel("District")  
plt.ylabel("Vaccination Rate (%)") plt.show()
```

